

CCAR-P Domain 3: Exam Playbook

Key learnings from the Kaveri dock lab, written for the exam room. Domain 3 (Integration) is 19 percent of the paper, about 12 of 63 questions, the heaviest domain. Every stem is some version of the distance between a Friday demo that worked and a Tuesday at 40,000 calls a day.

1. What a Domain 3 stem is really asking

The Professional exam's verbs are identify, evaluate, justify. A stem gives you a scenario with two or three numbers and four options that all sound competent. The question underneath is always one of these:

- **Which machine?** lookup, aggregate, or retrieval, decided by the shape of the data (3.6).
- **Carry it or fetch it?** cache a small stable always-used set, retrieve anything else (3.5, 3.8).
- **Which dial, and does the budget hold?** an addition compared at p95 against an SLA or a cost ceiling (3.3).
- **Who may open this door, and is it written down?** scopes in code, denials logged (3.2).
- **Which primitive?** tool, resource, prompt, or an agent-to-agent brief (3.7).
- **What should ride on the belt?** trim by logs and consequence, catalogue the rest (3.1).
- **What would you see, and when?** trace ids, structured logs, p95 not mean, categories not ids (3.4).

Name the question underneath before you read the options. Most wrong options answer a different question competently.

2. The six-step method for any stem

1. **Find the hop.** Which of the eight hops is the stem standing in? Door, router, gate, connector, shelves, belt and window, dials, cameras. The objective number follows from the hop.
2. **Find the shape.** If numbers are present, which of the eight arithmetic shapes is it? Belt tax, blended cost, p95 plus addition, skip-RAG threshold, cache-read equivalents, batch feasibility, overlap ratio, cardinality.
3. **Do the sum in the margin.** Ten seconds. Combine exactly two of the three numbers given. The third is usually a decoy (the mean beside a p95, a per-schema size beside two count_tokens readings).
4. **Say the verdict word.** fits or breaches, trim or keep, cache or retrieve, lookup or RAG. The verdict is what the option letter tests; the number is how you got there without guessing.
5. **Check the hop before.** The correct option usually respects the hop upstream of the one the stem names. A dials question is decided by what the shelves can deliver; a belt question by what the cameras logged.
6. **Eliminate by family.** Each wrong option belongs to a distractor family. Tag it, drop it. If two options survive, prefer the one that changes less: the over-engineered option is wrong more often than the cautious one.

3. Distractor families, and the sentence that gives each away

Code	Family	How it reads in a stem	Session example
OE	over-engineered	'for consistency', 'to unify', 'so that every asset is handled by one mechanism'	wrapping a single-consumer endpoint in MCP; RAG over a 38k rate card
VG	vocabulary gap	a right-sounding term for the wrong thing	mean reported as health; 'SSE transport'; situating 'runs on every query'
TR	trade-off reversed	pays a cost and names the wrong benefit	largest model everywhere at 9.40 vs a 6 ceiling; mid-tier on the refund path
CM	constraint missed	ignores a number the stem gave you	rerank inside a 2 s path at p95 1.9; a 2,000-token prefix on a 4,096 minimum
HL	human loop skipped	automates a judgement call	a negotiating agent exposed as book_slot(id) -> slot
OP	over-permissive	'to simplify operations', 'one identity', 'union of scopes'	kaveri-svc; chatbot with sop:read over escalation contacts
SF	silent failure	'to reduce log volume', composing a figure from text	denials not logged; RAG asked 'how many'
SD	sounds diligent	activity in place of mechanism	permissions in the system prompt; dashboard shipped with launch; a pilot promoted as-is

The exam punishes over-flagging as much as under-flagging. Six decisions in the Wrong Tower looked wrong and were right: one server for four clients, a rarely-used escalation tool kept, SSO plus API keys, rotation itself, one embedding model, caching the preamble. An option that 'fixes' a correct decision is OE.

4. Key learnings by objective

Each card: what the objective tests, the rule to apply, the number to carry, and the tell that marks the wrong option.

3.1 The belt: tool catalogue `e2_bloat_audit.py`

- **Rule:** trim what the logs never use, keep what consequence demands even if rare, catalogue what is used rarely and fetch its schema on choice.
- **Number:** belt tax = idle schemas x tokens per schema x calls per day. Nine tools at ~193 tokens each is ~1,740 per call; six idle on 40,000 calls is 60M tokens a day. Two count_tokens readings: subtract them.
- **Second effect:** lookalike tools (two ETA tools with one description) raise the wrong-tool rate. Merge or sharpen the docstring; the docstring is the interface.
- **Tell:** 'a schema costs only ~260 tokens' (ignores the multiplier); 'cheap insurance' (OE); a tool with two calls dropped (escalate_to_human is kept by consequence).

3.2 The gate: authorisation `e3_gate_pass.py`

- **Rule:** authorize, then audit_log, then the tool body, always in that order. Scopes checked in code keyed on the caller, never in the system prompt. Log every decision, denials most of all.
- **Identity:** two travel together. The agent's service account is what the gate checks; the human user is propagated for per-dispatcher rules and attribution. One shared identity holding the union of scopes kills both.
- **Wildcards:** rotating a credential does not narrow it. A valid token with scope * is the Refund Heist's ghost. Narrow to the one scope the job needs. Least privilege applies to resources too.
- **Tell:** 'to simplify operations' (OP); 'the prompt states you must never call' (SD); 'denied calls are not logged to reduce volume' (SF); 'scope will be reviewed in a later phase' (CM).

3.3 The dials: cost, latency, model tier no script; the 47-paise bill in One Ticket

- **Rule:** every dial has a number behind it. Write the justification as constraint + dial + direction + number: 'drafts err at ~4 rupees and are 55 percent of volume, so mid-tier; 4 s SLA holds at p95 3.1 s; blended 4.90 under 6.'
- **Blended cost:** sum of (share x price), compared with the ceiling before touching a dial. 'All routes' means share 1.00, so the blend is just the price.
- **Latency:** current p95 + the addition, compared with the SLA at p95. The mean is never in the sum. Fitting is necessary, not sufficient: each addition must buy measured accuracy (rerank yes, k=40 no).
- **Degradation:** fan out concurrently (wall clock = slowest scout), 3 s timeout, one retry, then a cached value labelled stale. Defaults are not decisions.
- **Tell:** 'mean 310 ms is well within the 3 s SLA' (VG); 'largest model everywhere since accuracy matters' (TR); 'no timeout so the dispatcher always gets the freshest value' (CM); 'top-5 because it is the default' (SD).

3.4 The cameras: observability `e7_dock_cameras.py`

- **Rule:** mint one trace id at the door, before anything is answered, and stamp it on every stage including across worker boundaries. One structured JSON line per stage with ms, tokens, cost. Report p95, never the mean.
- **Cardinality:** categories (stage, carrier, route, tier) are metric labels; ids (trace, consignment) live in logs and traces. Labels multiply; ids explode (4 stages x 40,000 ids = 160,000 series a day; a 10 percent sample is still an id).
- **Alerting:** few, dear, customer-felt symptoms (p95 breach, error rate, queue depth). Alerting on every anomaly is the same as no alerting.
- **The walk:** alert -> dashboard names the stage -> metric says since when -> trace shows the slow span -> log line names the request. Remove a layer and the next step is a guess.
- **Tell:** 'reported as mean' (VG); 'a metric per consignment id for drill-down' (OE); 'journeys reconstructed by timestamp correlation' (SF); 'dashboard delivered in the launch sprint' (SD). Observability gaps do not error; they wait.

3.5 The shelves: chunking and indexing `e4_chunking.py`, `e5_hybrid.py`

- **Rule:** chunk along meaning. Heading-aware cuts, modest overlap, the title and version written inside the chunk text (situated so both indexes see it and citations carry a version. Fixed cuts are how a cancellation chunk answers a re-filing question.
- **Hybrid:** keyword (BM25) and vector are two halves, each covering the other's blind spot. Vector-only fails on exact codes and Hindi-English terms; keyword-only fails on synonyms. Fuse, then rerank where the SLA can afford ~200 ms. Top-20 beat 5 and 10.
- **Timing:** retrieval quality is decided at shelving time, not query time. Situating runs once at ingest (adds nothing to the query path, cuts failures 35 to 49 percent). A weekly corpus needs a weekly re-index.
- **Numbers:** overlap / chunk = duplicated fraction; index grows ~1 + ratio. 400 over 500 is 80 percent duplicated, too far; 40 over 350 is 11 percent, seam insurance.
- **Tell:** 'fixed 500 tokens for uniform performance' (CM); 'keyword search is legacy' (VG); 'labels in a separate metadata table for cleanliness' (CM); 'contextual retrieval runs a model call on every query' (VG).

3.6 The shape: retrieval router `e6_shape_router.py`

- **Rule:** retrieval follows the shape of the data. A keyed record (consignment id, HSN code) wants a structured lookup: the answer, not things like it. A count or trend wants an aggregate query. Procedure prose wants hybrid RAG. A cheap regex in front, checked in order, decides.
- **Why it is a law:** RAG on an id turns certainty into probability. RAG asked 'how many' composes a fluent wrong number, the worst failure class. A question can carry two shapes; split it and send each half to its machine.
- **Law 3 outranks Law 5:** a 12k-row tariff table proposed for the cached prefix is neither cache nor RAG; keyed rows want a lookup tool. The data's shape picks the machine before the context strategy is a question.
- **Tell:** 'embed the TMS rows so status is answered from similar chunks' (OE); 'let the model compose the figure' (SF); 'handle exact codes by raising the similarity threshold' (TR).

3.7 The connector: MCP `e1_kaveri_mcp.py`, `e1_probe.py`

- **Rule:** tools act, resources inform, prompts template. Reading a document is a resource read into context with no tool call spent; a reusable procedure is a prompt versioned once on the server; a tool is for side effects.
- **When MCP earns its place:** many clients and a changing tool list. One server for KAVI, the tower, the UI and the IDE is the home ground. One client, one owner, no churn: the direct call was fine; wrap it when a second client appears.
- **Vocabulary (current spec):** transports are stdio and Streamable HTTP. Sampling is deprecated; integrate the model directly. A remote transport stem asks for both the mechanism and the auth: transport decides authN, the scope table decides authZ, per-team tokens scoped and rotated.
- **Agents are not tools:** a far side with judgement (a carrier's slot-negotiating agent) is agent-to-agent: a brief, authority limits, a budget, replies treated as proposals.
- **Tell:** 'exposed as tools so every read is in the audit log' (CM); 'migrated behind MCP for consistency' (OE); 'MCP SSE' (VG); 'a stateless tool book_slot(id) -> slot' (HL/CM).

3.8 The window: context strategy `e8_two_bills.py`

- **Rule:** map first, territory on demand. A lean per-route belt, a cached preamble, retrieved chunks, the question. The monolith (all schemas plus the whole corpus) spends 362k tokens on insurance for a question that needs one tool and one section.
- **The honest case:** small (under ~200k tokens, ~500 pages at ~400 tokens a page), stable, and always used means carry it all with caching, no retriever, one round-trip. All three conditions; 150k read by 30 percent of calls is retrieve.
- **Cache arithmetic:** cached reads ~0.1x. Prefix must clear the tier minimum (1,024 to 4,096; Haiku 4.5 is 4,096) or it never caches. Any byte change invalidates the prefix from that point; clearing old tool results is such a change, so clear in one big block (clear_at_least) and save essentials to the memory tool first.
- **Long runs:** server-side context editing (clear_tool_uses, clear_thinking, exclude_tools) replaces client-side summarisation. Clearing deletes; compaction summarises; they are different features.
- **Tell:** 'full corpus in every request, offset by caching' (OE/VG); 'a stable always-used pack fetched progressively as best practice' (TR); a 2,000-token preamble 'marked for caching' on Haiku (CM); batch feasibility judged by time before checking the window (CM).

5. The five laws, as decision rules

Law	Decision rule in the exam room	Depends on
1 Every tool is a door (3.1, 3.2)	Nothing on the belt without a scope and a log line; nothing off the belt that consequence requires.	the cameras knowing which tools were called
2 Chunk along meaning (3.5)	Heading cuts, situated, hybrid, rerank where affordable, re-index at corpus cadence.	the corpus being big enough to need RAG
3 Retrieval follows shape (3.6)	Id -> lookup. Count -> aggregate. Prose -> RAG. Shape picks the machine before any context strategy.	a cheap router in front
4 You cannot fix what you cannot see (3.4)	Trace id at the door, JSON per stage, p95 not mean, categories not ids, few dear alerts.	being built before launch, not with it
5 Map first, territory on demand (3.8)	Catalogue plus top-k unless the set is small, stable and always used; then cache the monolith.	a connector that lists before it calls

Read the middle column of the book's own table twice: each law is only possible because of another. That is why stems mix objectives.

6. The arithmetic card

Shape	Expression	Verdict words	Decoy to ignore
Belt tax (3.1)	idle x tokens/schema x calls/day; or (count_A - count_B) x calls	trim / keep	per-schema size when two counts are given
Blended cost (3.3)	sum(share x price) vs ceiling	fits / breaches	the expensive tier's price before weighting
p95 + addition (3.3/3.4)	p95 + added ms vs SLA	fits / breaches at p95	the mean
Skip-RAG (3.5/3.8)	pages x ~400 vs ~200k; and stable; and always used	cache / retrieve	size alone
Cache reads (3.8)	cached x ~0.1; hit-rate weighted; prefix >= tier minimum	token-equivalents / never caches	a token count without the round-trip
Batch feasibility (3.8)	fits a window? then latency x cases vs window	fits / retrieve first	time before window
Overlap (3.5)	overlap / chunk; index ~ 1 + ratio	seam insurance / too far	chunk count
Cardinality (3.4)	product of distinct label values	fine / the trap	'only a sample of ids'

7. Six crossover stems, solved in the method

Stem (objectives)	Hop, shape, sum	Verdict and why
Denials panel spikes for kavi-agent on process_refund from the drafting route; latency normal; no refund went out (3.2 + 3.4)	Gate and cameras. No sum. The gate worked and the register fed the panel.	Not an attack, not noise: a belt problem. Remove the tool from that route's belt, leave the scope denied, keep the panel.
4 s SLA at p95 3.85 s; proposal adds contextual retrieval at ingest to cut failures 49 percent (3.5 + 3.3)	Shelves and dials. p95 + addition: situating adds 0 ms at query time.	Accept. It runs at ingest, weekly. Reranking (+200 ms) would breach; situating does not. Know where each dial's cost lands.
A 12k-row tariff table proposed for the cached prefix for HSN lookups (3.6 + 3.8)	Shape before window. Keyed rows.	Neither cache nor RAG: a structured lookup tool. Stuffing turns exact answers into recall. Law 3 outranks Law 5.
Three teams reach the SOP server over Streamable HTTP; stem asks mechanism and auth (3.7 + 3.2)	Connector and gate. Many clients, changing list.	One MCP server, OAuth per-team tokens scoped sop:read and rotated. Transport decides authN; the table decides authZ.
30 tools across aggregated servers; any request uses at most 3; wrong-tool rate rising (3.1 + 3.8)	Belt and window. 7.8k standing tokens.	Per-route default belt plus a one-line catalogue fetched on choice: ~1.5k standing tokens and the lookalikes leave the belt.
Leadership calls retrieve healthy at mean 310 ms; SLA 3 s end to end; p95 2.9 s (3.4 + 3.3)	Cameras then dials. 2.9 + draft 1.2 = 4.1 > 3 at p95.	The SLA is a promise about the tail; 5 percent already live at 2.9 s in retrieval alone. Timeout plus cached ETA on the cold-carrier tail. Do not touch the mean.

8. Sentences that should make you reach for the pen

The calm sentence	What it hides	Obj
'validated in a two-week pilot with one team, promote as-is'	the demo-vs-dock trap; nothing was under load	all
'to simplify operations, one shared service identity'	attribution dies, least privilege impossible	3.2
'the prompt states you must never call process_refund'	instruction is not configuration	3.2
'denied calls are not logged, to reduce log volume'	the most interesting lines are gone	3.2
'for consistency, every asset handled by one mechanism'	the shape of the data was ignored	3.6 / 3.8
'keyword search is a legacy technique'	half of hybrid thrown away	3.5
'the pilot mean of 310 ms is well within the SLA'	the tail is where the SLA lives	3.4 / 3.3
'no timeout, so the dispatcher always gets the freshest value'	a cold API owns the whole SLA	3.3
'prompt caching will offset the cost' of the full corpus	1.4M tokens fits no window	3.8
'as current best practice recommends' (progressive fetch of a 45k always-used pack)	best practice reversed; small stable always-used is cached	3.8
'a dashboard delivered in the same sprint as launch'	no baseline; instrumentation comes first	3.4
'top-5 because it is the vector database's default'	defaults are not decisions	3.3 / 3.5

9. The one-minute incident walk (for any 'what would you look at first' stem)

Alert fires on a customer-felt symptom -> dashboard names the stage -> metric shows since when -> trace shows which span lost the time -> log line names the request, carrier and consignment. Five layers, one minute, no guessing. If a stem removes a layer, the answer is the step that becomes a guess, and the time to discovery is 'much later'.

10. Exam-day checklist

1. Read the stem once for the story, once for the numbers. Underline every number and ask which two combine.
2. Name the hop and the objective before reading the options.
3. If a mean and a p95 are both given, the mean is the decoy. If two count_tokens readings are given, subtract.
4. Compute in the margin, write the verdict word, then read the options looking for that word.
5. Check the upstream hop: does the option respect what the shelves, cameras or gate already decided?
6. Tag each rejected option with its family. If you cannot tag it, you have not found why it is wrong.
7. Prefer the option that changes less. 'Fixing' a correct decision is OE and costs a false alarm.
8. A remote-transport stem asks for mechanism and auth. A denials stem is usually a belt problem. A 'how many' stem is never RAG.
9. Vocabulary check: Streamable HTTP not SSE; resource not tool for documents; clearing not compaction; p95 not mean; categories not ids.
10. Write any justification as constraint + dial + direction + number.

Every flaw in the Wrong Tower was one of the eight parts' traps wearing a suit. Every sum in the Margin was already printed by a repo script. The exam does not test new material; it tests whether you recognise the Tuesday failure hiding inside the Friday sentence.

Companions in the repository: D3_Session_Summary.pdf (what was done), e1 to e9 (runnable evidence), the book volumes. Next sitting: Dock Book II, 45 stems, 90 minutes, cold. Re-sit the Wrong Tower in a week with the overlay hidden; target 27+ with at most one false alarm.